

Solomonoff Induction is not the paragon of generalization

Kaarel Hänni

August 2025

Abstract

For any probability distribution over infinite bitstrings, a Solomonoff Inductor is roughly at least as good at predicting next bits of the bitstring as any computable predictor P — more precisely, the total log loss of the Solomonoff Inductor cannot be worse than that of P by more than a constant c_P (independent of the number of bits predicted and of the distribution on sequences). One might want to summarize this canonical result as “you can’t make a predictive model that generalizes from data better than Solomonoff Induction”. In this paper, I argue that this is misguided: that while Solomonoff Induction indeed (roughly) beats every computable inductor at predicting raw data, it is nevertheless quite possible for a predictive model to generalize from data better than Solomonoff Induction, because generalization quality is often importantly different from the ability to predict raw data. While this point is perhaps fairly obvious once stated, I think it is easy to miss; in this paper, I spell out the point in detail. I also discuss some ideal predictive models that can generalize better than Solomonoff Induction.

1 plan

things to include:

- a human can totally handle new situations well even when SI outputs would not
- formally, there’s a trivial case where we have some computable criterion that outputs are supposed to meet. and you can just make an inductor whose outputs keep meeting the computable criterion, when those of SI might not. one should construct a concrete example here where SI keeps taking Ls
- ok but you could say this is sort of silly. i’d say it is mathematically silly, but nevertheless philosophically important to understand!
- define a property inductor. this sometimes works well, and feels less silly. it arguably captures the intuitive idea of “please continue like this” better than SI. its approximations are computable things which can do better than SI. construct an example where it does better than SI
- Is it possible to beat SI at providing binary labels to data? if we care about individual labels being good in a log loss additive way, then no i guess
- is this just AIXI with some utility function?

2 Introduction

The following setup is standard in machine learning.

- One has a *training data set* $\mathcal{D} \subseteq X \times Y$ — that is, a finite set $\mathcal{D} = \{(x_i, y_i) : i \in I\}$ of things $x_i \in X$ (“inputs”) with associated things $y_i \in Y$ (“outputs” or “labels”).¹
 - An example: we could have each x_i be either a picture of a cat or a picture of a dog, with each label y_i being either “cat” or “dog” correspondingly.

¹Technically, we will assume (perhaps non-standardly for machine learning, but arguably with minimal loss of generality) that all x_i and y_i are ultimately given as finite bitstrings; when we speak of them as being other kinds of things — e.g., digital texts or sound sequences or video sequences, or elements of some finite set of labels — we’re implicitly assuming that some simple way to encode these other kinds of things as bitstrings has been fixed. (Alternatively, one could go with the convention that x_i and y_i are ultimately natural numbers — this would be equivalent for our purposes.)

– Another example: we could have

$$\mathcal{D} = \{(n, \pi(n)) : 1 \leq n \leq 11\} = \{(1, 0), (2, 1), (3, 2), (4, 2), (5, 3), (6, 3), (7, 4), (8, 4), (9, 4), (10, 4), (11, 5)\},$$

where $\pi(n)$ is the prime counting function.

- One wishes to use this training data set $\mathcal{D}$ to make a machine M that assigns labels to inputs x and that does so *well* — in particular, M should assign labels well even to inputs x which do not show up in $\mathcal{D}$. To be precise, we will say that M should be a Turing machine and

There is a very natural ideal thing of this kind

3 mess

so, we are interested in examples where computable things keep beating solomonoff. i guess they must not be predicting stuff well then? they must be not making

why can't we just keep doing ood shifts, and keep seeing solomonoff fail? well maybe we could, but then a computable thing which is getting things right must also keep failing at prediction